

Improving Money-Maker With Player-Level Data

Executive summary

I inspected the repository structure and the MLB-related training, inference, and ingestion code directly from the public GitHub source. The repo is a Python-only, multi-asset project with sports modules for MLB, NBA, soccer, EPL, and World Cup tooling, but the only validated game-outcome artifact I found is the MLB moneyline model at `alpha/models/mlb_win_probability.pkl` / `.meta.json`. That MLB model is trained on just eight team-state features — Elo difference, team win percentages, run differential, rest days, and a home flag — with no player identities, no confirmed lineups, no true starting-pitcher quality, no bullpen workload, and no Statcast-derived contact or stuff metrics. The current released artifact reports 0.549 accuracy on 1,705 games with Brier score 0.2456, only modestly better than trivial coin-flip and home-rate baselines. ¹

The highest-leverage accuracy upgrade is not “more generic team stats.” It is replacing the current team-only pregame row with a player-conditioned game state built from the confirmed or projected starting pitcher, projected lineup strength, bullpen availability, and injury-adjusted lineup absences. In MLB, starting-pitcher effectiveness, lineup efficiency, and pitcher removal / bullpen strategy are central determinants of run prevention and run scoring, while batting-order position by itself has only a small effect relative to who is actually in the lineup. That combination makes player inclusion and player quality much more important than the current repo’s team-level-only snapshot. ²

If your goal is **maximize accuracy and win percentage**, not EV, then the best path is to build a leak-proof MLB moneyline pipeline around four feature blocks: probable-starter quality and rest; lineup talent and platoon-adjusted matchup quality; bullpen freshness / relief depth; and injury / absence deltas relative to normal starting strength. Then evaluate it with **date-based walk-forward retraining**, separate calibration windows, and confidence-gated pick emission so you can report both overall accuracy and high-confidence win rate. ³

My bottom-line recommendation is to keep the current eight-feature model only as a baseline. Build a new **player-aware LightGBM moneyline model** on top of official MLB Stats API, official MLB lineup / probable-pitcher / injury pages, official Baseball Savant Statcast metrics, and Retrosheet historical lineups and play-by-play for backfill. Implement ablations in this order: starter-only, starter+lineup, starter+lineup+bullpen, then full model. That gives you the clearest path to higher win rate with the least wasted engineering time. ⁴

What the repo currently does

At the top level, the repository contains a central `alpha` package, a large `scripts` directory, and additional folders for soccer and MLB work such as `ProphitBet-Soccer-Bets-Predictor` and `mlb_outcomes`. The `pyproject.toml` describes the project as a “Unified multi-asset trading engine: stocks, crypto, sports,” and the dependency set includes `scikit-learn`, `MLB-StatsAPI`,

`pybaseball`, `nba_api`, and `soccerdata`. The sports engine tree includes MLB, NBA, EPL, soccer, props, and World Cup modules. ⁵

For the task you asked about, the most relevant files are concentrated in `scripts/`, `alpha/engines/sports/`, `alpha/data/ingestion/`, and `alpha/models/`. The table below summarizes the key files I inspected and what each one currently contributes. The mapping is taken directly from the repo's directory listings and source files. ⁶

File	Current role
<code>scripts/train_mlb_moneyline.py</code>	Fetches completed MLB games, builds chronological pregame rows, fits logistic and HistGradientBoosting candidates, calibrates with Platt scaling, and writes the model bundle.
<code>alpha/engines/sports/mlb_training.py</code>	Defines the MLB moneyline feature schema and leakage-safe team-state pregame row builder.
<code>alpha/models/mlb_win_probability.meta.json</code>	Stores the released artifact metadata, feature names, split dates, and metrics.
<code>alpha/engines/sports/mlb_model.py</code>	Live inference wrapper; loads the validated bundle if available, otherwise falls back to older team-stat / pitcher-adjusted logic.
<code>alpha/data/ingestion/mlb_stats.py</code>	Pulls MLB team batting / pitching stats, player batting / pitching stats, probable pitchers, and today's schedule.
<code>alpha/data/ingestion/mlb_injuries.py</code>	Pulls MLB injuries through ESPN-based endpoints and aggregates simple batting loss impact.
<code>alpha/engines/sports/mlb_prop_model.py</code>	MLB player-prop model; currently uses season-average rows and synthetic rolling values, not true historical game logs.
<code>scripts/evaluate_moneyline_walkforward.py</code>	Walk-forward reporting scaffold, but written around NBA paper-bet logging rather than a full historical MLB retrain-per-fold pipeline.

The current validated MLB training schema is extremely small. `mlb_training.py` defines `FEATURE_NAMES` as just `elo_diff`, `home_win_pct`, `away_win_pct`, `home_run_diff`, `away_run_diff`, `home_rest_days`, `away_rest_days`, and `home_indicator`. The training script sorts games chronologically, uses a simple 60/20/20 train-calibration-test split, fits logistic regression and HistGradientBoosting, calibrates on the middle block with a one-dimensional logistic regression on log-odds, and selects the candidate with the best Brier score. ⁷

The released artifact confirms that the selected candidate was `hist_gradient_boosting`, validated against trivial baselines, with reported accuracy of 54.90%, log loss 0.6842, and Brier score 0.2456 on 1,705

test games from August 19, 2025 through June 18, 2026. This is a real signal, but it is still very close to baseline and leaves a lot of headroom for player-aware modeling. ⁸

One of the most important repo-specific findings is that the validated MLB bundle and the fallback MLB code are not the same system. The validated bundle uses only the eight team-state features above. By contrast, the fallback path in `mlb_model.py` builds a much wider hand-crafted feature vector from team batting stats, team pitching stats, crude injury adjustments, probable-pitcher adjustments, and even decimalized odds fields. Since you asked to ignore odds-related code and prioritize accuracy / win percentage, the fallback path should not be the basis for the next iteration; it should be treated as a legacy branch. ⁹

Another important finding is that the prop layer is not suitable as-is for player-level moneyline enrichment. `MLBPropModel` fetches season-average batter or pitcher rows from `get_batter_stats()` / `get_pitcher_stats()`, extracts a single base statistic, and then creates synthetic rolling values with `_synthetic_values(base_val, player_name)`. That means the current prop code is not using true historical player game logs, which makes it the wrong place to borrow lineup-strength or recent-form signals from if you care about accuracy. ¹⁰

The player-level gaps that matter most

The most important gap is **starting-pitcher quality at the player level**. The validated schema does not include the probable or confirmed starter at all. The fallback code does try to look up probable pitchers and apply a crude “ace ERA boost” or bullpen ERA fallback, but that is still a blunt heuristic and it is not part of the released validated bundle. In MLB, starting-pitcher quality and the timing of removal to the bullpen have direct implications for win probability, and MLB’s own advanced stats definitions show why: starting-pitcher quality is closely tied to runs, hits, HR, BB, strikeouts, times through the order, and endurance, while Statcast exposes directly useful pitcher rest and times-through-order fields. ¹¹

The next highest-value gap is **actual lineup quality and lineup absences**. The current model never asks who the nine hitters are. That is a major omission, because the official MLB ecosystem provides probable pitchers, starting lineups, and injury-report updates, and modern baseball simulation work shows that lineup efficiency and player-specific event probabilities drive run distributions. At the same time, classic batting-order research found that exact batting-order arrangement has only a small effect relative to the talent in the lineup itself. In practical terms: model the people, their handedness, and their matchup quality; do not spend your first sprint overfitting slot-3 versus slot-4 batting order one-hots. ¹²

The third major gap is **bullpen availability and recent workload**. Your current team rest-day feature is too coarse for MLB. Team rest is not the same as bullpen freshness, and the game state is often decided by which high-leverage relievers are actually available after the starter exits. Official Statcast CSV fields include `pitcher_days_since_prev_game`, `n_thruorder_pitcher`, release metrics, and pitch characteristics that can be rolled into better starter and relief workload features. The Sloan bullpen-strategy paper also reinforces that starter removal and relief usage strategy are old but central baseball optimization questions, not edge-case details. ¹³

The fourth gap is **true Statcast skill instead of noisy surface stat lines**. `mlb_stats.py` currently ingests only basic team stats and very simple player stats such as AVG, OBP, SLG, HR/game, ERA, WHIP, and K/9.

Official Baseball Savant provides much richer measurements: xBA, xwOBA, xERA, exit velocity, hard-hit rate, barrels, launch-angle sweet-spot rate, bat speed, fast-swing rate, spin rate, release extension, spin axis, and effective speed. Those are the best raw materials for player-level quality estimation because they are closer to underlying contact and pitch quality than raw batting average or ERA alone. ¹⁴

The fifth gap is **injuries and availability being modeled too weakly**. The repo's MLB injury ingestion aggregates only batting average lost and HR rate lost for absent players, which underweights OBP, SLG, contact-quality loss, speed, defensive loss, and all pitcher injuries. There is also a concrete reliability risk in the static ESPN fallback map: it appears to assign the same ESPN team ID value ²¹ to both the Milwaukee Brewers and the New York Mets, which could corrupt fallback injury lookups if dynamic discovery fails. For accuracy-focused modeling, injuries need to become structured player-availability features, not one crude batting-average penalty. ¹⁵

The table below is the priority order I would use for MLB moneyline accuracy. This ranking is based on the repo's current omissions, official MLB data availability, and the baseball research above. ¹⁶

Player feature block	Examples of concrete fields	Why it matters most	Expected impact
Starting-pitcher quality and rest	xERA, xwOBA allowed, K rate, BB rate, release extension, spin rate, effective speed, days since previous game, recent pitch count	The starter shapes the first half of the game and determines how quickly weak bullpen arms get exposed	Very high
Lineup strength versus handedness	Confirmed 1–9 hitters, batter handedness, xwOBA, xBA, hard-hit rate, barrel rate, bat speed, sprint speed, platoon splits	Captures who is actually available to score runs against tonight's starter	Very high
Bullpen availability	Recent appearances, pitches / batters faced last 1/3/5 days, top reliever availability, inherited-runner performance proxy	Team “rest days” miss the true post-starter state of the game	High
Injury and transaction deltas	IL status, day-to-day status, expected return, player removed from usual lineup, recent call-up / option	Makes lineup-strength estimates respond to real roster changes	High
Defense and catcher quality	OAA, fielding run value, framing / blocking proxies, catcher starter ID	Helps on run prevention, stolen-base prevention, and pitcher outcomes	Medium
Baserunning quality	Sprint speed, stolen-base attempt rate, extra-base-taken rate proxies	Improves conversion from contact to runs in close matchups	Medium

Player feature block	Examples of concrete fields	Why it matters most	Expected impact
Exact batting-order slot effects	Slot index, top-3 / top-6 concentration	Useful, but materially less important than who is in the lineup	Low-to-medium

Recommended data sources and the exact fields to fetch

If you want primary or near-primary sources, the best stack is **MLB Stats API + MLB.com lineup / probable-pitcher / injury pages + Baseball Savant Statcast + Retrosheet + Chadwick ID mapping**. The repo already depends on `MLB-StatsAPI` and `pybaseball`, but for the next version I would treat `pybaseball` as a convenience wrapper and make the canonical dataset come from official MLB / Baseball Savant / Retrosheet sources first, with Chadwick as the ID bridge. ¹⁷

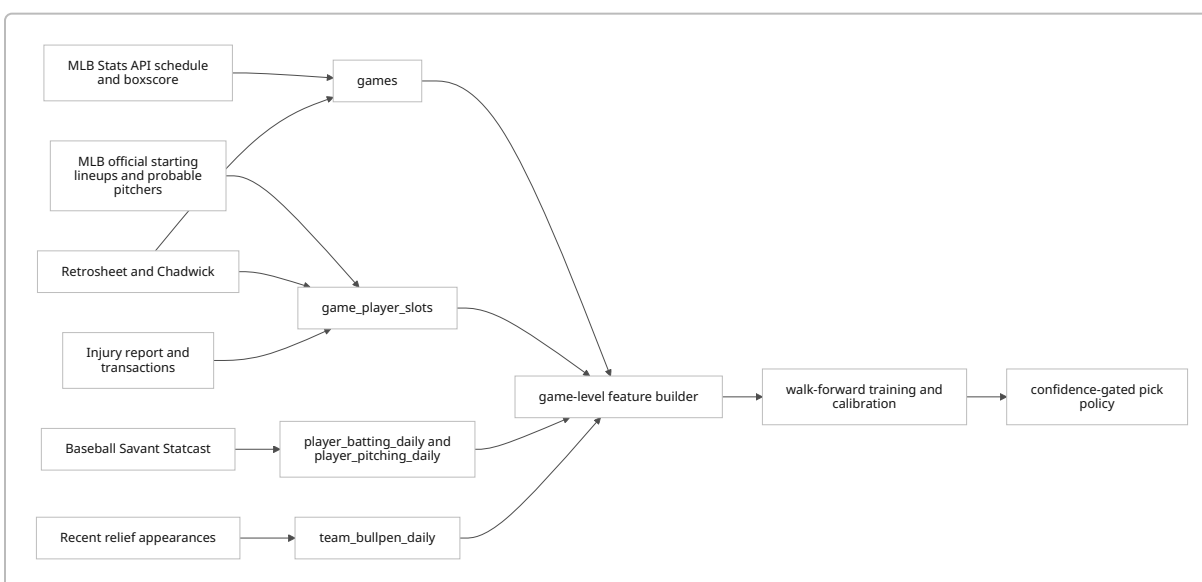
The source plan below is the one I would implement.

Priority	Source	What to use it for	Exact fields to fetch
Highest	MLB Stats API and official MLB pages	Schedule, game IDs, probable starters, lineups, roster state	<code>gamePk</code> , <code>gameDate</code> , <code>home_team</code> , <code>away_team</code> , <code>venue</code> , <code>status</code> , <code>home_probable_pitcher</code> , <code>away_probable_pitcher</code> , confirmed lineup player IDs, batting order, field positions, active roster status, doubleheader flags, series/game number ¹⁸
Highest	Baseball Savant / Statcast	Player skill and matchup quality	For hitters: <code>estimated_woba_using_speedangle</code> , <code>estimated_ba_using_speedangle</code> , <code>launch_speed</code> , <code>launch_angle</code> , barrel / hard-hit / sweet-spot context, bat speed, fast swing rate, sprint speed. For pitchers: <code>release_spin</code> , <code>release_extension</code> , <code>effective_speed</code> , <code>spin_axis</code> , <code>xERA</code> , <code>pitch_name</code> , <code>n_thruorder_pitcher</code> , <code>pitcher_days_since_prev_game</code> ¹⁹
High	MLB official injury report pages	Day-of availability and return timing	player name, injury type, IL date, expected return, status text, team, pitcher / hitter distinction ²⁰
High	Retrosheet event files and parsed play-by-play	Historical lineups, starters, substitutions, game context for backfill	lineup “start” rows, player ID, home/away side, batting order position, fielding position, game logs, parsed play-by-play rows, annual rosters ²¹

Priority	Source	What to use it for	Exact fields to fetch
High	Chadwick Register	Stable ID joins across MLBAM / Retrosheet / other systems	<code>key_mlbam</code> , <code>key_retro</code> , <code>key_bbref</code> , <code>key_fangraphs</code> , player names and aliases ²²
Fallback only	ESPN injuries and current repo <code>pybaseball</code> pulls	Backup when official MLB injury / lineup pages are unavailable	Do not treat as ground truth if the official source is available; fix the static team-ID fallback first ²³

A practical way to structure the warehouse is to create five canonical tables: `games`, `game_player_slots`, `player_batting_daily`, `player_pitching_daily`, and `team_bullpen_daily`. `games` should hold one row per game with IDs, teams, date, venue, and starter IDs. `game_player_slots` should hold the lineup and starter roster for each game and team. The daily player tables should contain only **as-of-date rolled features**, never target-game outcomes. `team_bullpen_daily` should summarize all relief appearances up to, but not including, the target first pitch. That table design matches what MLB, Savant, and Retrosheet expose and makes leakage control much easier. ²⁴

This is the architecture I would use.



The repo already has the skeleton for this change: it knows how to fetch schedule data, probable pitchers, player stats, and injuries. What it lacks is the historical player-game backbone and the model schema that consumes it. ²⁵

How to engineer the new features without leakage

The most important engineering rule is simple: **every player feature must be “known before first pitch.”** The repo already uses that philosophy in `mlb_training.py` and, even more explicitly, in the NBA `prop_features.py` module, which says target-game minutes and stats must never influence row inclusion or features, and optional features should stay missing rather than being silently filled with constants. Keep that exact philosophy for the new MLB moneyline builder. ²⁶

For hitters, I would build one daily row per player with shifted rolling features at multiple horizons, for example 7, 14, and 30 days or 5, 10, and 20 games. The core batting features should be Statcast-first: rolling xwOBA, xBA, hard-hit rate, barrel rate, average launch speed, sprint speed, and a platoon-adjusted split versus tonight’s starter handedness. Then aggregate those player rows into lineup-level features such as `lineup_mean_xwoba`, `top5_mean_xwoba`, `lineup_mean_barrel_rate`, `lefty_lineup_share`, `bench_gap_to_starters`, and `injury_adjusted_missing_lineup_value`. Because batting-order arrangement alone has a small long-run effect, I would keep batting-order weighting modest and put most of the model capacity into lineup talent and platoon matchups. ²⁷

For starting pitchers, use two groups of features. First, long-horizon skill: rolling xERA, xwOBA allowed, strikeout and walk rates, bat-missing or chase proxies if you have them, pitch-mix concentration, spin rate, extension, and effective speed. Second, short-horizon freshness and context: days since last appearance, pitch count in last outing, pitch count over last 7 and 14 days, velocity change versus recent baseline, and whether the team is likely to limit pitch count because of a short-rehab or return-from-IL context. The official Savant docs expose several of the relevant physical and workload-adjacent fields directly. ²⁸

For bullpens, do not just average team ERA. Build a relief-availability layer. At minimum, track relief pitchers’ batters faced and pitches over the prior 1, 3, and 5 days; whether the team’s top two or three relievers pitched the last day; and a team-level weighted bullpen quality score that uses only relievers likely to be available tonight. If you cannot yet build a reliever-availability model, a blunt but much better baseline than the current setup is `weighted_relief_xERA_available_last3d` and `high_leverage_pitch_count_last3d`. This will matter far more than a generic team rest feature. ²⁹

For injuries and missing data, use **shrinkage and flags**, not flat fills. If a batter has only 40 recent plate appearances, shrink his rolling xwOBA toward team and league averages weighted by recent PA. If a rookie lacks Statcast history, backfill with team-position averages plus an explicit `is_rookie_sparse_history` flag. If a lineup is unconfirmed early in the day, either use a projected lineup with a `lineup_unconfirmed=1` flag or, if maximizing win rate is more important than coverage, do not emit a pick until the lineup is confirmed on the official page. That abstention policy is especially valuable for MLB because official starting lineups are available day-of and last-minute scratches materially change lineup quality. ³⁰

On leakage prevention, I would impose these hard rules:

1. Every rolling player or team feature must be computed with a `shift(1)` so the target game is never in its own history.
2. For same-day doubleheaders, do not allow game two to consume game one outcomes unless your intended prediction time is explicitly between games.

3. Separate **training**, **calibration**, and **test** by date. Do not fit the calibrator on the same rows used to fit the base model.
4. Use date-based walk-forward splits with a one-day gap, or a custom event-time splitter that mimics `TimeSeriesSplit`'s expanding-window behavior.
5. Never use outcome-derived Savant fields like post-play score changes or target-game run-expectancy deltas as features. ³¹

A good default split design is **expanding-window seasons**, not one fixed 60/20/20 split.

`TimeSeriesSplit` exists exactly because random CV on time-ordered data leaks future information and produces misleading optimism; it keeps splits time-ordered, lets training sets expand over time, and supports a `gap` parameter. Your current repo already respects chronology at row generation time, but the next step is to make evaluation chronology-respecting too. ³²

Experiment design for accuracy and win percentage

The right way to do this is with **ablations**, not one giant all-in model. Since the repo's current released MLB model is simple, you can get a very clean read on where the gains come from if you add feature blocks in layers. I would run the following ladder: current v1.3 baseline; team-only expanded baseline without any odds features; starter-only model; starter+lineup model; starter+lineup+bullpen; full model with injuries and defense. That makes it obvious whether lineup engineering is worth more than bullpen engineering, and it will keep you from shipping a bigger model that is harder to maintain but not materially better. ³³

For model families, I would keep three:

- **Regularized logistic regression** on differential features as the interpretable baseline.
- **HistGradientBoostingClassifier** because it is already in the repo and performed best among current candidates.
- **LightGBM** as the likely best performer once you add richer nonlinear player interactions. LightGBM's key complexity controls are `num_leaves`, `max_depth`, `min_data_in_leaf`, sampling fractions, and regularization. ³⁴

For LightGBM specifically, the search space I would use first is:

- `n_estimators` : 300, 700, 1200
- `learning_rate` : 0.01, 0.03, 0.05
- `num_leaves` : 15, 31, 63
- `max_depth` : 4, 6, -1
- `min_data_in_leaf` : 50, 100, 200
- `feature_fraction` : 0.6, 0.8, 1.0
- `bagging_fraction` : 0.6, 0.8, 1.0
- `lambda_l1` : 0, 1, 5
- `lambda_l2` : 0, 1, 5

Those are not magic values; they are practical ranges that match the controls LightGBM documents for complexity and overfitting. ³⁵

For HistGradientBoosting, I would search:

- `max_depth` : 3, 4, 6
- `learning_rate` : 0.02, 0.05, 0.10
- `max_iter` : 300, 600, 1000
- `min_samples_leaf` : 20, 50, 100
- `l2_regularization` : 0, 0.1, 1, 5

For logistic regression, standardize numeric differentials and search `C` over 0.03, 0.1, 0.3, 1.0 with L2 penalty. Then compare sigmoid and isotonic calibration on a separate validation block; scikit-learn's calibration docs explicitly support these calibration modes, and a separate calibration stage is appropriate when you care about thresholded decision quality. ³⁶

Because you care about accuracy and win rate, I would use **two primary scorecards**:

1. **All-games pick accuracy**: choose the higher-probability side every game and measure straight-up hit rate.
2. **Selective win rate**: emit a pick only when `abs(p_home - 0.5)` exceeds a threshold and the lineup/starter uncertainty flags are off.

That second scorecard is the one that most directly aligns with your “win percentage over EV” preference. It is an inference from the repo's setting rather than a published rule, but it is the right one: if you do not need action on every game, you should expect a confidence-gated subset to beat the all-games hit rate. Pair that with Brier score, log loss, calibration tables, and coverage rate so you know whether the win-rate lift is real or just caused by predicting fewer games. ³⁷

For sample sizes, do not use the repo's 300-game minimum as your real standard. That threshold is only enough to keep the training script from crashing. For a serious player-level MLB moneyline model, use at least **three full seasons** for training and validation, and test in **month-sized or season-fragment blocks** large enough to include several hundred games per fold. MLB's 162-game, 30-team regular season gives you plenty of data volume for this. If you use bat-tracking features like bat speed or blast rate, note that Savant says those metrics are available from 2023 forward, with 2023 being partial. ³⁸

For statistical tests, use a paired framework:

- **McNemar's test** on paired correct / incorrect decisions for model A versus model B.
- **Diebold-Mariano** on per-game loss differentials for Brier or log-loss comparisons.
- **Block bootstrap by date or series** for confidence intervals around selective win rate and all-games accuracy.
- **Permutation test score** if you want an additional nonparametric check that the observed cross-validated score improvement is meaningful. ³⁹

This is the ablation ladder I would run first.

Experiment	Features added	Model(s)	Main question
Baseline current	Existing 8 team-state features	Logistic, HistGB	Reproduce current artifact
Team-only expanded	Current team stats, no odds	Logistic, HistGB, LightGBM	Is richer team context enough?
Starter-only	Add probable-starter quality and rest	Same	How much of the lift comes from pitchers alone?
Starter plus lineup	Add confirmed / projected lineup aggregates and platoon	Same	Is lineup identity the biggest missing block?
Starter plus lineup plus bullpen	Add relief-availability features	Same	Does bullpen freshness add stable gain?
Full player-aware	Add injuries, defensive / catcher context, speed	Same	Is there meaningful incremental lift after the top three blocks?

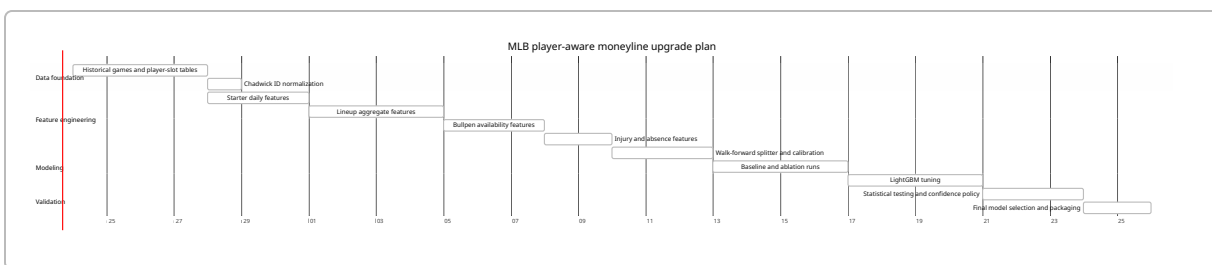
A prioritized roadmap and the code to start with

The most valuable implementation choice is to be ruthless about scope. Do not start by trying to rebuild the whole sports engine. Start by replacing the MLB moneyline row builder. Everything else should support that goal. The roadmap below is sequenced by expected accuracy impact and engineering risk. It is designed to get you to a materially better MLB moneyline model quickly, without entangling yourself in the repo's odds and prop branches. [40](#)

Priority	Task	Estimated effort	Why this goes first
Highest	Build historical <code>games</code> + <code>game_player_slots</code> tables from MLB / Retrosheet / Chadwick	2–4 days	Everything else depends on correct player-game joins
Highest	Add starter-level daily features and swap them into the moneyline row builder	2–3 days	Fastest likely accuracy gain
Highest	Add lineup aggregates versus starter handedness	3–5 days	Biggest missing offensive signal
High	Add bullpen-availability layer	2–4 days	Major late-game signal not captured by team rest
High	Replace crude injury penalties with structured absence features; fix fallback map	1–2 days	Important and a concrete source-quality fix

Priority	Task	Estimated effort	Why this goes first
High	Implement date-based walk-forward retraining + separate calibration	2-3 days	Prevents deceptive evaluation
Medium	Add defense / catcher / baserunning context	2-4 days	Likely additive, but not the first-order gain
Medium	Add abstention / confidence policy and reporting	1-2 days	Converts better probabilities into better win rate

This is the experiment schedule I would use.



The first code change I would make is a new ingestion layer for **game-player slots**. The current repo knows the schedule and probable pitchers, but not the historical lineup / starter roster in a modeling-friendly schema. The code below shows the intended shape.

```
from __future__ import annotations

import pandas as pd
import statsapi

def fetch_schedule_range(start_date: str, end_date: str) -> pd.DataFrame:
    games = statsapi.schedule(start_date=start_date, end_date=end_date)
    rows = []
    for g in games:
        if g.get("status") not in {"Final", "Scheduled", "Pre-Game"}:
            continue
        rows.append(
            {
                "game_id": str(g["game_id"]),
                "game_date": str(g["game_date"])[0:10],
                "home_team": g["home_name"],
                "away_team": g["away_name"],
                "home_probable_pitcher": g.get("home_probable_pitcher"),
                "away_probable_pitcher": g.get("away_probable_pitcher"),
                "venue": g.get("venue_name"),
            }
        )
```

```

        }
    )
    return pd.DataFrame(rows)

def normalize_lineup_rows(raw_lineups: list[dict]) -> pd.DataFrame:
    """
    Expected output:
    game_id, team, side, batting_slot, player_id, player_name, bats, throws,
    position, confirmed
    """
    return pd.DataFrame(raw_lineups)

```

The second code change is the daily player feature store. The key pattern is `groupby(...).shift(1)` before any rolling or exponential window, so the target game never contaminates its own predictors. That is the same leakage-safe idea your repo already uses in simpler form. ²⁶

```

import pandas as pd

BATTING_ROLL_COLS = [
    "xwoba",
    "xba",
    "barrel_rate",
    "hard_hit_rate",
    "avg_launch_speed",
]

def build_batter_daily_features(game_logs: pd.DataFrame) -> pd.DataFrame:
    """
    game_logs columns:
    player_id, game_date, opponent_pitcher_hand, pa, xwoba, xba,
    barrel_rate, hard_hit_rate, avg_launch_speed, sprint_speed
    """
    df = game_logs.sort_values(["player_id", "game_date"]).copy()

    for col in BATTING_ROLL_COLS:
        shifted = df.groupby("player_id")[col].shift(1)
        df[f"{col}_ewm10"] = shifted.groupby(df["player_id"]).transform(
            lambda s: s.ewm(span=10, adjust=False, min_periods=5).mean()
        )
        df[f"{col}_roll20"] = shifted.groupby(df["player_id"]).transform(
            lambda s: s.rolling(20, min_periods=5).mean()
        )

    df["pa_last20"] = (
        df.groupby("player_id")
        ["pa"].shift(1).groupby(df["player_id"]).transform(

```

```

        lambda s: s.rolling(20, min_periods=1).sum()
    )
)

# simple empirical-Bayes shrinkage
league_xwoba = df["xwoba"].mean()
weight = (df["pa_last20"] / (df["pa_last20"] + 100)).fillna(0.0)
df["xwoba_shrunk"] = weight * df["xwoba_roll20"] + (1 - weight) *
league_xwoba

return df

```

The third code change is the **game-level feature builder**. This is where you turn starters, lineups, injuries, and bullpen state into one row per game.

```

import numpy as np
import pandas as pd

def aggregate_lineup_strength(lineup_df: pd.DataFrame) -> pd.DataFrame:
    # lineup_df: one row per batter in a game lineup
    agg = (
        lineup_df.groupby(["game_id", "team", "side"], as_index=False)
        .agg(
            lineup_mean_xwoba=("xwoba_shrunk", "mean"),
            top6_mean_xwoba=("xwoba_shrunk", lambda s: s.nlargest(6).mean()),
            lineup_mean_barrel=("barrel_rate_roll20", "mean"),
            lineup_mean_speed=("sprint_speed", "mean"),
            lefty_share=("bats", lambda s: np.mean(pd.Series(s).eq("L"))),
            missing_batter_features=("xwoba_shrunk", lambda s: s.isna().sum()),
        )
    )
    return agg

def build_game_matrix(
    games: pd.DataFrame,
    home_lineup: pd.DataFrame,
    away_lineup: pd.DataFrame,
    starters: pd.DataFrame,
    bullpen_daily: pd.DataFrame,
) -> pd.DataFrame:
    X = games.merge(home_lineup, on=["game_id"], how="left", suffixes=("",
    "_home"))
    X = X.merge(away_lineup, on=["game_id"], how="left", suffixes=("", "_away"))
    X = X.merge(starters.add_prefix("home_sp_"),
    left_on="home_probable_pitcher", right_on="home_sp_player_name", how="left")
    X = X.merge(starters.add_prefix("away_sp_"),

```

```

left_on="away_probable_pitcher", right_on="away_sp_player_name", how="left")
    X = X.merge(bullpen_daily.add_prefix("home_bp_"), left_on=["home_team",
"game_date"], right_on=["home_bp_team", "home_bp_game_date"], how="left")
    X = X.merge(bullpen_daily.add_prefix("away_bp_"), left_on=["away_team",
"game_date"], right_on=["away_bp_team", "away_bp_game_date"], how="left")

    X["sp_xera_diff"] = X["home_sp_xera_roll20"] - X["away_sp_xera_roll20"]
    X["lineup_xwoba_diff"] = X["lineup_mean_xwoba"] -
X["lineup_mean_xwoba_away"]
    X["bullpen_avail_diff"] = X["home_bp_available_quality"] -
X["away_bp_available_quality"]
    X["home_indicator"] = 1.0
    return X

```

For training, I would move away from the repo's single chronological split into a walk-forward loop with a calibration block inside each fold. This is the piece that best aligns with your actual deployment setting. 41

```

from __future__ import annotations

from dataclasses import dataclass
from typing import Iterable

import lightgbm as lgb
import numpy as np
import pandas as pd
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import accuracy_score, brier_score_loss, log_loss

@dataclass
class FoldResult:
    fold_name: str
    accuracy: float
    brier: float
    logloss: float
    coverage: float

def walkforward_splits(df: pd.DataFrame) -> Iterable[tuple[np.ndarray,
np.ndarray, np.ndarray]]:
    """
    Returns train_idx, calib_idx, test_idx
    Example policy:
    - train through season N-1
    - calibrate first 30 days of season N
    - test next 30 days of season N
    """
    dates = pd.to_datetime(df["game_date"]).sort_values().unique()

```

```

# replace with your real monthly/season logic
for i in range(180, len(dates) - 60, 30):
    train_dates = dates[: i - 30]
    calib_dates = dates[i - 30 : i]
    test_dates = dates[i : i + 30]

    train_idx = df.index[df["game_date"].isin(train_dates)]
    calib_idx = df.index[df["game_date"].isin(calib_dates)]
    test_idx = df.index[df["game_date"].isin(test_dates)]
    yield train_idx.to_numpy(), calib_idx.to_numpy(), test_idx.to_numpy()

def fit_and_score(df: pd.DataFrame, feature_cols: list[str], target_col: str) -
> list[FoldResult]:
    results: list[FoldResult] = []

    for fold_no, (tr, cal, te) in enumerate(walkforward_splits(df), start=1):
        Xtr, ytr = df.loc[tr, feature_cols], df.loc[tr, target_col]
        Xcal, ycal = df.loc[cal, feature_cols], df.loc[cal, target_col]
        Xte, yte = df.loc[te, feature_cols], df.loc[te, target_col]

        base_model = lgb.LGBMClassifier(
            objective="binary",
            n_estimators=700,
            learning_rate=0.03,
            num_leaves=31,
            max_depth=6,
            min_data_in_leaf=100,
            subsample=0.8,
            colsample_bytree=0.8,
            reg_alpha=1.0,
            reg_lambda=1.0,
            random_state=42,
        )
        base_model.fit(Xtr, ytr)

        # calibrate on future-only calibration block
        calibrator = CalibratedClassifierCV(base_model, method="sigmoid",
cv="prefit")
        calibrator.fit(Xcal, ycal)

        proba = calibrator.predict_proba(Xte)[: , 1]
        pred = (proba >= 0.5).astype(int)

        results.append(
            FoldResult(
                fold_name=f"fold_{fold_no}",
                accuracy=accuracy_score(yte, pred),
                brier=brier_score_loss(yte, proba),

```

```

        logloss=log_loss(yte, proba),
        coverage=1.0,
    )
)

return results

```

Finally, there are three repo-specific fixes I would make immediately, even before full retraining:

First, replace `FEATURE_NAMES` in `alpha/engines/sports/mlb_training.py` with a new player-aware schema and rename the current eight-feature version as `MLB_FEATURES_V1_BASELINE`. That preserves backward compatibility while making the new model explicit. ⁴²

Second, quarantine the synthetic `_synthetic_values()` path in `MLBPropModel` so no one accidentally reuses it for moneyline or lineup engineering. If you keep the prop model, rewrite it on real game logs. ¹⁰

Third, fix the injury fallback mapping and then stop using “avg_lost / hr_lost” as the main absence encoding. Use player IDs and lineup availability instead. ¹⁵

The overall recommendation is straightforward: your intuition about incorporating individual player data is correct, but the winning move is not “add a few player averages.” It is to change the unit of information in the MLB moneyline model from **team snapshot** to **tonight’s actual player-conditioned matchup**. In this repo, that means confirmed or projected starters, confirmed or projected lineups, bullpen availability, and structured injuries — all built into leak-proof, date-ordered training and evaluated with walk-forward calibration and paired statistical tests. That is the most concrete, highest-probability path to meaningfully better accuracy and win percentage in this codebase. ⁴³

¹ ⁵ <https://github.com/Minkeejung0415/money-maker>

<https://github.com/Minkeejung0415/money-maker>

² ²⁹ <https://www.sloansportsconference.com/research-papers/bullpen-strategies-for-major-league-baseball>

<https://www.sloansportsconference.com/research-papers/bullpen-strategies-for-major-league-baseball>

³ <https://www.mlb.com/probable-pitchers>

<https://www.mlb.com/probable-pitchers>

⁴ <https://docs.statsapi.mlb.com/login>

<https://docs.statsapi.mlb.com/login>

⁶ https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html

⁷ ¹³ ¹⁶ ¹⁷ ²⁶ ³¹ ⁴⁰ ⁴² ⁴³ <https://github.com/Minkeejung0415/money-maker/blob/main/pyproject.toml>

<https://github.com/Minkeejung0415/money-maker/blob/main/pyproject.toml>

8 33 37 https://raw.githubusercontent.com/Minkeejung0415/money-maker/main/alpha/models/mlb_win_probability.meta.json
https://raw.githubusercontent.com/Minkeejung0415/money-maker/main/alpha/models/mlb_win_probability.meta.json

9 11 https://raw.githubusercontent.com/Minkeejung0415/money-maker/main/alpha/engines/sports/mlb_model.py
https://raw.githubusercontent.com/Minkeejung0415/money-maker/main/alpha/engines/sports/mlb_model.py

10 https://raw.githubusercontent.com/Minkeejung0415/money-maker/main/alpha/engines/sports/mlb_prop_model.py
https://raw.githubusercontent.com/Minkeejung0415/money-maker/main/alpha/engines/sports/mlb_prop_model.py

12 30 <https://www.mlb.com/starting-lineups>
<https://www.mlb.com/starting-lineups>

14 25 https://raw.githubusercontent.com/Minkeejung0415/money-maker/main/alpha/data/ingestion/mlb_stats.py
https://raw.githubusercontent.com/Minkeejung0415/money-maker/main/alpha/data/ingestion/mlb_stats.py

15 23 https://raw.githubusercontent.com/Minkeejung0415/money-maker/main/alpha/data/ingestion/mlb_injuries.py
https://raw.githubusercontent.com/Minkeejung0415/money-maker/main/alpha/data/ingestion/mlb_injuries.py

18 24 <https://github.com/toddrob99/MLB-StatsAPI/wiki/Function%3A-schedule>
<https://github.com/toddrob99/MLB-StatsAPI/wiki/Function%3A-schedule>

19 27 https://baseballsavant.mlb.com/leaderboard/expected_statistics
https://baseballsavant.mlb.com/leaderboard/expected_statistics

20 <https://www.mlb.com/injury-report>
<https://www.mlb.com/injury-report>

21 <https://www.retrosheet.org/eventfile.htm>
<https://www.retrosheet.org/eventfile.htm>

22 <https://github.com/chadwickbureau/register>
<https://github.com/chadwickbureau/register>

28 <https://baseballsavant.mlb.com/csv-docs>
<https://baseballsavant.mlb.com/csv-docs>

32 34 38 41 https://raw.githubusercontent.com/Minkeejung0415/money-maker/main/scripts/train_mlb_moneyline.py
https://raw.githubusercontent.com/Minkeejung0415/money-maker/main/scripts/train_mlb_moneyline.py

35 <https://lightgbm.readthedocs.io/en/latest/Parameters.html>
<https://lightgbm.readthedocs.io/en/latest/Parameters.html>

36 <https://scikit-learn.org/stable/modules/calibration.html>
<https://scikit-learn.org/stable/modules/calibration.html>

39 https://www.statsmodels.org/dev/generated/statsmodels.stats.contingency_tables.mcnemar.html
https://www.statsmodels.org/dev/generated/statsmodels.stats.contingency_tables.mcnemar.html